Resolución — Taller SQL sobre HR 2 — Agregación, GROUP BY y CASE

Bases de Datos I · resolución paso a paso

Antes de empezar

- Cada ejercicio muestra lo que se pide, la idea, los pasos, la consulta y un aviso de error común.
- Las consultas están escritas para Oracle Database 19c y solo leen el esquema HR.
- Hay más de una forma correcta de resolver casi todos los ejercicios; cuando conviene, se muestra una variante.

Ejercicio 1 — Cuántos hay y cuántos cobran comisión

Nivel: Básico · **Tablas:** `employees`

Se pide. En una sola consulta, mostrar cuántos empleados hay en total y cuántos de ellos tienen comisión.

Mostrar: el total de empleados y el total con comisión (una sola fila).

Idea. `COUNT(*)` cuenta filas; `COUNT(columna)` cuenta solo las filas donde esa columna no es `NULL`.

Pasos.

1. `COUNT(*)` cuenta todos los empleados.
2. `COUNT(e.commission_pct)` ignora los `NULL`, así que cuenta solo a quienes tienen comisión.
3. Sin `GROUP BY`, el resultado es una sola fila.

```
SELECT COUNT(*)               AS total_empleados,
       COUNT(e.commission_pct) AS con_comision
FROM hr.employees e;
```

Ojo. La diferencia entre las dos cuentas es justo la cantidad de empleados sin comisión.

Ejercicio 2 — Radiografía de la nómina

Nivel: Básico · **Tablas:** `employees`

Se pide. Buscar la suma de todos los salarios, el salario más alto, el más bajo, el promedio y la desviación estándar.

Mostrar: una sola fila con cinco columnas; el promedio y la desviación con dos decimales.

Idea. Las funciones de agregación resumen muchas filas en una sola.

Pasos.

1. `SUM(e.salary)` da la nómina total.
2. `MAX` y `MIN` dan los extremos.
3. `ROUND(AVG(e.salary), 2)` da el promedio con dos decimales.
4. `ROUND(STDDEV(e.salary), 2)` da la desviación estándar.
5. Sin `GROUP BY`, el resultado completo es una sola fila.

```
SELECT SUM(e.salary)                AS suma_salarios,
       MAX(e.salary)                AS salario_maximo,
       MIN(e.salary)                AS salario_minimo,
       ROUND(AVG(e.salary), 2)      AS promedio,
       ROUND(STDDEV(e.salary), 2)   AS desviacion_estandar
FROM hr.employees e;
```

Ojo. STDDEV en Oracle es la desviación estándar muestral; la poblacional es STDDEV_POP. Con 107 empleados la diferencia es pequeña, pero no es la misma cifra.

Ejercicio 3 — Extremos por departamento

Nivel: Intermedio · **Tablas:** `employees`, `departments`

Se pide. Mostrar, por departamento, el salario más alto y el más bajo.

Mostrar: nombre del departamento, salario máximo y salario mínimo, ordenado por nombre del departamento.

Idea. `GROUP BY` crea un grupo por cada departamento y las funciones de agregación se calculan dentro de cada grupo.

Pasos.

1. Se unen `hr.employees` y `hr.departments` por `department_id`.
2. `GROUP BY d.department_id, d.department_name` define los grupos.
3. `MAX(e.salary)` y `MIN(e.salary)` se calculan por grupo.
4. Se ordena por el nombre del departamento.

```
SELECT d.department_name,
       MAX(e.salary) AS maximo,
       MIN(e.salary) AS minimo
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
ORDER BY d.department_name;
```

Ojo. Toda columna del `SELECT` que no sea una función de agregación debe estar en el `GROUP BY`. Quien no tiene departamento no aparece: el `JOIN` interno lo descarta.

Ejercicio 4 — Departamentos con más de diez empleados

Nivel: Intermedio · **Tablas:** `employees`, `departments`

Se pide. Listar los departamentos que tengan más de 10 empleados.

Mostrar: nombre del departamento y cantidad de empleados, de mayor a menor.

Idea. `WHERE` filtra filas antes de agrupar; `HAVING` filtra grupos después de agrupar.

Pasos.

1. Se agrupa por departamento y se cuenta con `COUNT(*)`.
2. La condición recae sobre el resultado del conteo, así que va en `HAVING COUNT(*) > 10`.
3. Se ordena por el alias `empleados`, de mayor a menor.

```
SELECT d.department_name,
       COUNT(*) AS empleados
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
HAVING COUNT(*) > 10
ORDER BY empleados DESC;
```

Ojo. `WHERE COUNT(*) > 10` falla con `ORA-00934` (función de grupo no permitida aquí): las funciones de agregación no se pueden usar en el `WHERE`.

Ejercicio 5 — Los cinco cargos que más nómina concentran

Nivel: Intermedio · **Tablas:** `employees`, `jobs`

Se pide. Mostrar los cinco cargos con la mayor suma de salarios.

Mostrar: cargo (`job_title`), cantidad de empleados y suma de salarios.

Idea. Se agrupa por cargo, se ordena por la suma y se corta el resultado con `FETCH FIRST`.

Pasos.

1. Se unen `hr.employees` y `hr.jobs` por `job_id` para leer el título del cargo.
2. Se agrupa por cargo y se calculan `COUNT(*)` y `SUM(e.salary)`.
3. Se ordena de mayor a menor por la suma.
4. `FETCH FIRST 5 ROWS ONLY` se queda con los cinco primeros.

```
SELECT j.job_title,
       COUNT(*)      AS empleados,
       SUM(e.salary) AS nomina
FROM hr.employees e
JOIN hr.jobs j ON j.job_id = e.job_id
GROUP BY j.job_id, j.job_title
ORDER BY nomina DESC
FETCH FIRST 5 ROWS ONLY;
```

Ojo. Si el quinto puesto está empatado, `FETCH FIRST 5 ROWS ONLY` corta de forma arbitraria; con `FETCH FIRST 5 ROWS WITH TIES` salen todos los empatados.

Ejercicio 6 — Costo real con comisiones

Nivel: Intermedio · **Tablas:** `employees`, `departments`

Se pide. Calcular, para los departamentos Sales y Shipping, el costo mensual de la nómina con y sin comisiones. La comisión de cada empleado es su `commission_pct` multiplicado por su salario.

Mostrar: departamento, nómina base y nómina con comisiones (número entero).

Idea. `NVL(x, 0)` reemplaza un `NULL` por 0 para que la cuenta no se anule.

Pasos.

1. Se unen `hr.employees` y `hr.departments` y se filtra `department_name IN ('Sales', 'Shipping')`.
2. La nómina base es `SUM(e.salary)`.
3. Con comisión es `SUM(e.salary * (1 + NVL(e.commission_pct, 0)))`.
4. Se agrupa por departamento y se redondea la segunda columna.

```
SELECT d.department_name,
       SUM(e.salary) AS nomina_base,
       ROUND(SUM(e.salary * (1 + NVL(e.commission_pct, 0))))
         AS nomina_con_comision
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
WHERE d.department_name IN ('Sales', 'Shipping')
GROUP BY d.department_id, d.department_name
ORDER BY d.department_name;
```

Ojo. Sin NVL, `salary * (1 + NULL)` da NULL: quienes no tienen comisión dejarían de sumar su salario y la nómina con comisión saldría menor que la base.

Ejercicio 7 — Mensaje según el salario

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Traer el empleado, su salario y un mensaje según el monto: menor a 1000, «pobrecito»; menor a 3000, «sigue intentando»; menor a 5000, «ya casi»; mayor a 10000, «Buenos días ingeniero»; cualquier otro caso, «otros».

Mostrar: nombre, apellido, salario y mensaje, ordenados por salario.

Idea. CASE evalúa las condiciones de arriba abajo y se queda con la primera que sea verdadera.

Pasos.

1. Cada rango es un WHEN con su THEN.
2. El orden importa: «menor a 3000» solo llega a las filas que no fueron «menor a 1000», o sea de 1000 a 2999.
3. Lo que no cumple ninguna regla cae en el ELSE: los salarios de 5000 a 10000.
4. END cierra el CASE, y se le pone el alias `mensaje`.

```
SELECT e.first_name,
       e.last_name,
       e.salary,
       CASE
         WHEN e.salary < 1000  THEN 'pobrecito'
         WHEN e.salary < 3000  THEN 'sigue intentando'
         WHEN e.salary < 5000  THEN 'ya casi'
         WHEN e.salary > 10000 THEN 'Buenos días ingeniero'
         ELSE 'otros'
       END AS mensaje
FROM hr.employees e
ORDER BY e.salary;
```

Ojo. Si pones primero `WHEN e.salary < 5000`, todos los salarios bajos reciben «ya casi»: los rangos más angostos van primero. Los bordes también cuentan: 1000 es «sigue intentando», 3000 es «ya casi», y 5000 y 10000 caen en «otros».

Ejercicio 8 — Cuántos empleados hay de cada mensaje

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Con las mismas categorías del ejercicio anterior, contar cuántos empleados hay en cada mensaje.

Mostrar: mensaje y cantidad de empleados, de más a menos.

Idea. Para agrupar por una expresión calculada se envuelve la consulta en una vista en línea y se agrupa por su alias.

Pasos.

1. La consulta interna calcula el mensaje de cada empleado, con el mismo CASE.
2. La consulta externa trata ese resultado como una tabla (x) y agrupa por `x.mensaje`.
3. `COUNT(*)` cuenta cuántos empleados cayeron en cada mensaje.
4. Se ordena por la cantidad.

```
SELECT x.mensaje,
       COUNT(*) AS empleados
FROM (SELECT CASE
               WHEN e.salary < 1000  THEN 'pobrecito'
               WHEN e.salary < 3000  THEN 'sigue intentando'
               WHEN e.salary < 5000  THEN 'ya casi'
               WHEN e.salary > 10000 THEN 'Buenos días ingeniero'
               ELSE 'otros'
             END AS mensaje
      FROM hr.employees e) x
GROUP BY x.mensaje
ORDER BY empleados DESC;
```

Ojo. Oracle no deja usar un alias del SELECT en el `GROUP BY` de la misma consulta (`ORA-00904`). Se repite el CASE o se usa una vista en línea, como aquí. Y los alias de tabla (x) van sin AS.

Ejercicio 9 — Contrataciones por año

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Mostrar, por año de contratación, cuántos empleados entraron y su salario promedio, solo para los años con 10 o más contrataciones.

Mostrar: año, cantidad de contratados y salario promedio redondeado a entero; ordenado por año.

Idea. Se agrupa por una expresión (el año de la fecha) y se filtra el grupo con `HAVING`.

Pasos.

1. `EXTRACT(YEAR FROM e.hire_date)` saca el año de la fecha.
2. Se agrupa por esa misma expresión.
3. `HAVING COUNT(*) >= 10` deja solo los años con muchas contrataciones.
4. Se ordena por año.

```
SELECT EXTRACT(YEAR FROM e.hire_date) AS anio,
       COUNT(*)                       AS contratados,
       ROUND(AVG(e.salary))           AS salario_promedio
FROM hr.employees e
GROUP BY EXTRACT(YEAR FROM e.hire_date)
HAVING COUNT(*) >= 10
ORDER BY anio;
```

Ojo. En el `GROUP BY` se repite `EXTRACT(YEAR FROM e.hire_date)`; el alias `anio` solo sirve en el `ORDER BY`. Si ningún año alcanza el umbral, el resultado sale vacío, y es una respuesta válida.

Ejercicio 10 — El peso de cada departamento

Nivel: Avanzado · **Tablas:** `employees`, `departments`

Se pide. Mostrar, por departamento, cuántos empleados tiene y qué porcentaje de la plantilla total representa.

Mostrar: departamento, empleados y porcentaje con un decimal, de mayor a menor porcentaje.

Idea. Una subconsulta escalar (que devuelve un único valor) puede usarse como un número más dentro de una operación.

Pasos.

1. Se cuenta cuántos empleados tiene cada departamento.
2. El total de la plantilla sale de `(SELECT COUNT(*) FROM hr.employees)`.
3. El porcentaje es `COUNT(*) * 100 / total`, redondeado a un decimal.
4. Se ordena por porcentaje.

```
SELECT d.department_name,
       COUNT(*) AS empleados,
       ROUND(COUNT(*) * 100
             / (SELECT COUNT(*) FROM hr.employees), 1) AS porcentaje
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
ORDER BY porcentaje DESC;
```

Ojo. El total cuenta a todos los empleados, también a quien no tiene departamento: por eso los porcentajes suman un poco menos de 100.

Ejercicio 11 — Bien pagados y los demás

Nivel: Avanzado · **Tablas:** `employees`, `departments`

Se pide. Para cada departamento, mostrar cuántos empleados ganan más de 8000 («bien pagados») y cuántos ganan 8000 o menos. Mostrar solo los departamentos que tengan al menos un empleado bien pagado.

Mostrar: departamento, cantidad de bien pagados y cantidad de los demás, ordenado por bien pagados de mayor a menor y luego por nombre.

Idea. `COUNT` ignora los `NULL`, y un `CASE` sin `ELSE` devuelve `NULL` cuando no se cumple: contar un `CASE` es contar solo las filas que cumplen.

Pasos.

1. `COUNT(CASE WHEN e.salary > 8000 THEN 1 END)` cuenta solo a los bien pagados.
2. `COUNT(CASE WHEN e.salary <= 8000 THEN 1 END)` cuenta a los demás, con la condición opuesta.
3. `HAVING` repite el primer conteo con `>= 1` para descartar los departamentos sin bien pagados.
4. Se ordena por el primer conteo y por el nombre.

```
SELECT d.department_name,
       COUNT(CASE WHEN e.salary > 8000  THEN 1 END) AS bien_pagados,
       COUNT(CASE WHEN e.salary <= 8000 THEN 1 END) AS otros
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
HAVING COUNT(CASE WHEN e.salary > 8000 THEN 1 END) >= 1
ORDER BY bien_pagados DESC, d.department_name;
```

Ojo. Es equivalente a `SUM(CASE WHEN ... THEN 1 ELSE 0 END)`. Con `COUNT` el `ELSE` no se escribe; con `SUM` el `ELSE 0` es obligatorio.

Ejercicio 12 — Una lista en una sola celda

Nivel: Avanzado · **Tablas:** `employees`, `departments`

Se pide. Por cada departamento con menos de 7 empleados, mostrar la lista de apellidos de sus empleados separados por coma y ordenados alfabéticamente.

Mostrar: departamento, cantidad de empleados y la lista de apellidos, ordenado por nombre del departamento.

Idea. `LISTAGG(columna, separador)` une los valores de cada grupo en un solo texto.

Pasos.

1. Se unen `hr.employees` y `hr.departments` y se agrupa por departamento.
2. `LISTAGG(e.last_name, ', ')` concatena los apellidos del grupo.
3. `WITHIN GROUP (ORDER BY e.last_name)` define el orden dentro del texto.
4. `HAVING COUNT(*) < 7` limita el resultado a los departamentos pequeños.

```
SELECT d.department_name,
       COUNT(*) AS empleados,
       LISTAGG(e.last_name, ', ')
         WITHIN GROUP (ORDER BY e.last_name) AS apellidos
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
HAVING COUNT(*) < 7
ORDER BY d.department_name;
```

Ojo. El resultado de `LISTAGG` no puede pasar de 4000 bytes (`ORA-01489`). Con departamentos grandes hay que filtrar o usar `ON OVERFLOW TRUNCATE`.